

Segunda Prova de Linguagens de Programação  
- DCC024 -  
Ciência da Computação

Nome: \_\_\_\_\_

“Eu dou minha palavra de honra que não trapacearei neste exame.”

Número de matrícula: \_\_\_\_\_

As regras do jogo:

- A prova é sem consulta.
- Quando terminar, não entregue nada além do caderno de provas para o instrutor.
- Quando escrever código, a sintaxe correta é importante.
- Cada estudante tem direito a fazer uma pergunta ao instrutor durante a prova. Isso inclui “Quanto tempo falta para acabar a prova”. Para a pergunta: “Posso ir ao banheiro”, a resposta é sim (deixe o telefone celular sobre a mesa para ir ao banheiro).
- A prova termina uma hora e quarenta minutos após seu início.
- Seja honesto e lembre-se: **você deu sua palavra de honra.**

Alguns conselhos:

- Escreva sempre algo nas questões, a fim de ganhar algum crédito parcial.
- Se não entender a questão, e já tiver gasto sua pergunta, escreva a sua interpretação da questão junto à resposta.
- A prova não é difícil, ela é divertida, então aproveite!

Tabela 1: Pontos acumulados (para uso do instrutor)

| Questão 1 | Questão 2 | Questão 3 | Q. Extra |
|-----------|-----------|-----------|----------|
|           |           |           |          |

Questão Extra (0.5 Pontos): “\_\_\_\_\_ alguém me dê um coração, que este já não bate nem apanha.”

1. Nesta questão você deverá implementar um predicado **anagram** em Prolog. Para facilitar este esforço, sua tarefa será dividida nas três partes abaixo:

(a) (3 Pontos) Implemente um predicado **subtext(Text, SubText)**, que seja verdade quando **SubText** for uma sublista da lista **Text**. Por exemplo:

```
?- subtext([a, n, a], X).  
X = [] ;  
X = [a] ;  
X = [n] ;  
X = [n, a] ;  
X = [a] ;  
X = [a, a] ;  
X = [a, n] ;  
X = [a, n, a].
```

(b) (3 Pontos) Escreva um predicado **perm(T, P)**, que seja verdadeiro quando a lista **P** for uma permutação dos elementos da lista **T**. Por exemplo:

```
?- perm([a, n, a], T).  
T = [a, n, a] ;  
T = [a, a, n] ;  
T = [n, a, a] ;  
T = [n, a, a] ;  
T = [a, a, n] ;  
T = [a, n, a] ;
```

Se quiser, você pode usar **select**:

```
?- select(a, [a, b, c], [b, c]).  
true ;
```

(c) (4 Pontos) Escreva um predicado **anagram(Pattern, Text)**, que seja verdadeiro houver uma sublista da lista **Text** que seja uma permutação dos elementos da lista **Pattern**. Exemplo:

```
?- anagram([a, b, c, d], [b, f, a, d, b, c]).  
true ;  
true ;  
false. /* Duas respostas possiveis! */  
  
?- anagram([a, b, c, d], [d, b, f, a, d, b, c]).  
true ;  
true ;  
true ;  
true ;  
false. /* Quatro respostas possiveis! */
```

2. O encapsulamento de dados é um importante princípio de programação. Várias linguagens de programação, como **Java**, **C++** e **SML/NJ** suportam o encapsulamento de dados via sintaxe que permite a programadores definir símbolos visíveis fora dos espaços de nomes em que eles foram declarados.
- (a) (1 Ponto) Quais mecanismos de encapsulamento de dados são fornecidos pela linguagem **C++** ao programador para diferenciar a visibilidade de símbolos declarados no mesmo espaço de nomes (seja ele instanciável ou não)?
  
  
  
  
  
  
  
  
  
  
  - (b) (3 Pontos) Quais mecanismos de encapsulamento de dados são fornecidos pela linguagem **Python** ao programador para diferenciar a visibilidade de símbolos declarados no mesmo espaço de nomes (seja ele instanciável ou não)?
  
  
  
  
  
  
  
  
  
  
  - (c) (3 Pontos) Quais mecanismos de encapsulamento de dados são fornecidos pela linguagem **C** ao programador para diferenciar a visibilidade de símbolos declarados no mesmo espaço de nomes (seja ele instanciável ou não)?
  
  
  
  
  
  
  
  
  
  
  - (d) (3 Pontos) Quais mecanismos de encapsulamento de dados são fornecidos pela linguagem **SML/NJ** ao programador para diferenciar a visibilidade de símbolos declarados no mesmo espaço de nomes (seja ele instanciável ou não)?

3. Uma função recursiva é dita de *cauda rasa* quando a última coisa que ela faz é chamar-se recursivamente. Re-implemente cada um dos predicados abaixo, para que eles sejam predicados de cauda rasa:

(a) (3 Pontos) O predicado que soma os elementos de uma lista:

```
sum([], 0).  
sum([H|T], X) :- sum(T, XAux), X is XAux + H.
```

(b) (3 Pontos) O predicado que inverte os elementos de uma lista:

```
myappend([], L, L).  
myappend([H|T], L, [H|LAux]) :- myappend(T, L, LAux).  
  
myreverse([], []).  
myreverse([H|T], R) :- myreverse(T, RT), myappend(RT, [H], R).
```

(c) (4 Pontos) Qual a vantagem de implementarmos funções de cauda rasa?